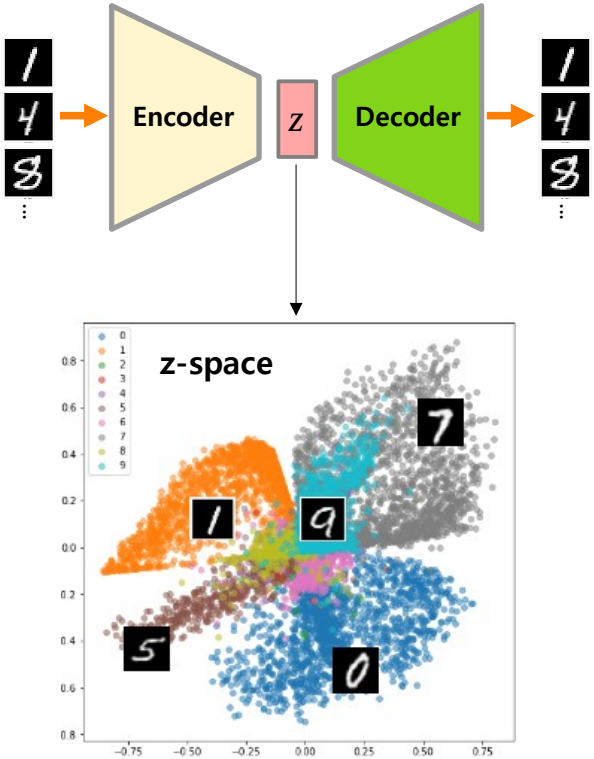


13. Autoencoder

Part 1: Dimensionality reduction

This video was produced in Korean and translated into English, and the audio was generated by AI (TTS).

www.youtube.com/@meanxai

Autoencoder. All rights reserved by www.youtube.com/@meanxai

MX-AI

[MXDL-13-01]

1. Overview of autoencoder

2. Dimensionality reduction

2-1. The basic structure of an autoencoder model
2-2. Implementation of an autoencoder model for reducing the dimensionality of MNIST images
2-3. The structure of a CNN-autoencoder model
2-4. 2D transposed convolutional layer
2-5. Implementation of an CNN-autoencoder model

[MXDL-13-02]

3. Noise reduction

3-1. Structure of an autoencoder model for noise reduction.
3-2. Denoising fashion MNIST images using a CNN-autoencoder
3-3. Deblurring fashion-MNIST images using a CNN-autoencoder
3-4. Denoising time series using a LSTM-autoencoder

[MXDL-13-03]

4. Sparse autoencoder

4-1. Overview of a sparse autoencoder model
4-2. Imposing a sparsity constraint on an autoencoder
4-3. L1 and KL divergence activity regularization
4-4. Implementation of a sparse autoencoder model to detect edges of MNIST images.

[MXDL-13-04]

5. Anomaly detection

5-1. Credit card transaction dataset
5-2. Preprocessing the dataset for fraud detection
5-3. implementation of an autoencoder model for credit card fraud detection.

[MXDL-13-05]

6. Variational Autoencoder (VAE)

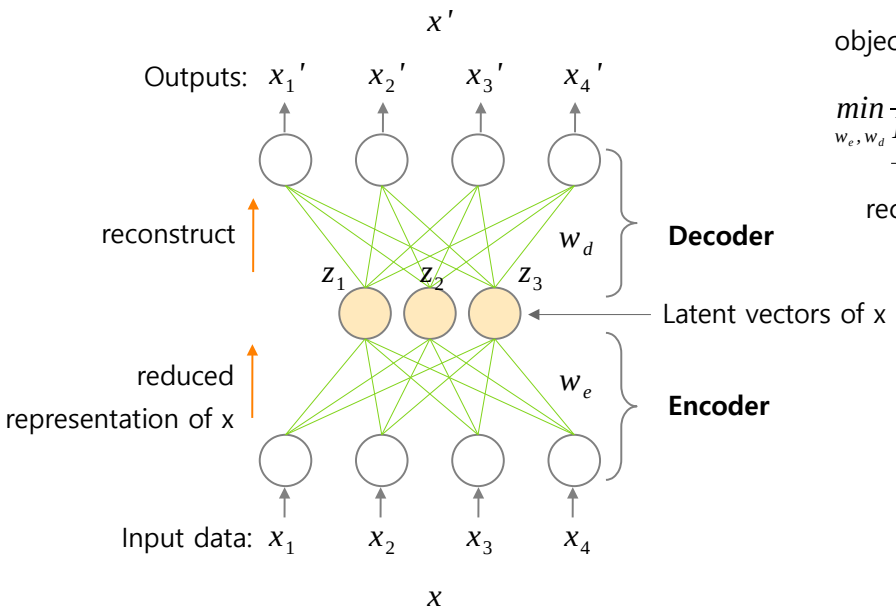
6-1. Overview of variational autoencoder
6-2. Structure of the variational autoencoder network
6-3. Loss function for VAE

[MXDL-13-06] → 6-4. Implementation of a VAE model for generating MNIST images

Autoencoder. All rights reserved by www.youtube.com/@meanxai

Overview of an autoencoder

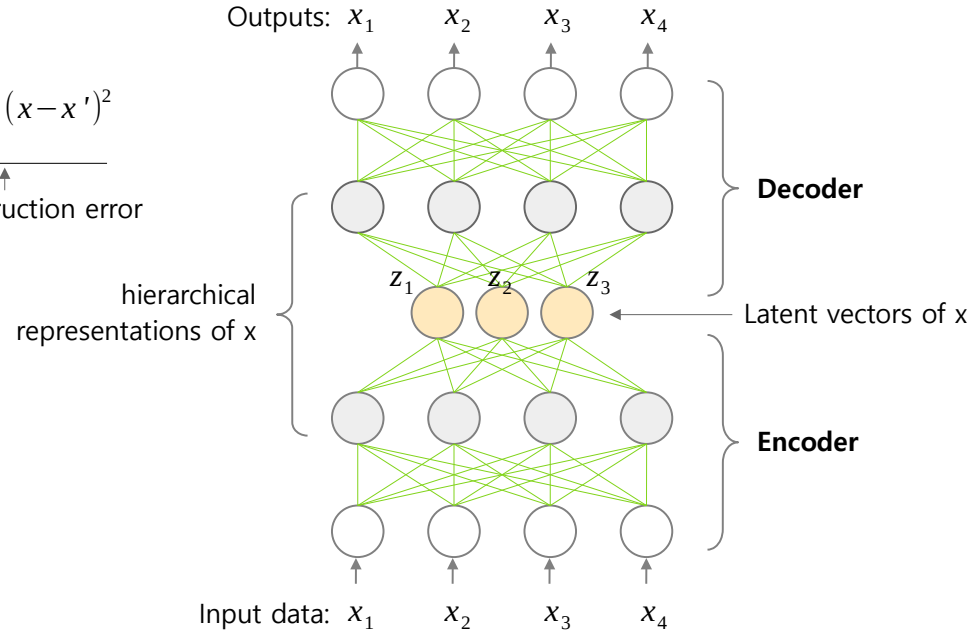
- An autoencoder is a network trained so that its output is as similar as possible to the input data, as shown in the figure below.
- An autoencoder neural network is an unsupervised learning algorithm that applies backpropagation.
- Autoencoders have two main components: an encoder and a decoder. The encoder typically reduces the dimensionality of the input data x to produce a latent vector z. The decoder reconstructs the original data x from the latent vector z, which is a reduced representation of x.
- An autoencoder can be configured as a single layer, as shown in the left figure below, or as multiple layers, called a stacked autoencoder, as shown in the right figure.
- Autoencoders can be applied to (1) dimensionality reduction, (2) noise removal, (3) anomaly detection, (4) generative models, etc.



objective:

$$\min_{w_e, w_d} \frac{1}{N} \sum (x - x')^2$$

reconstruction error

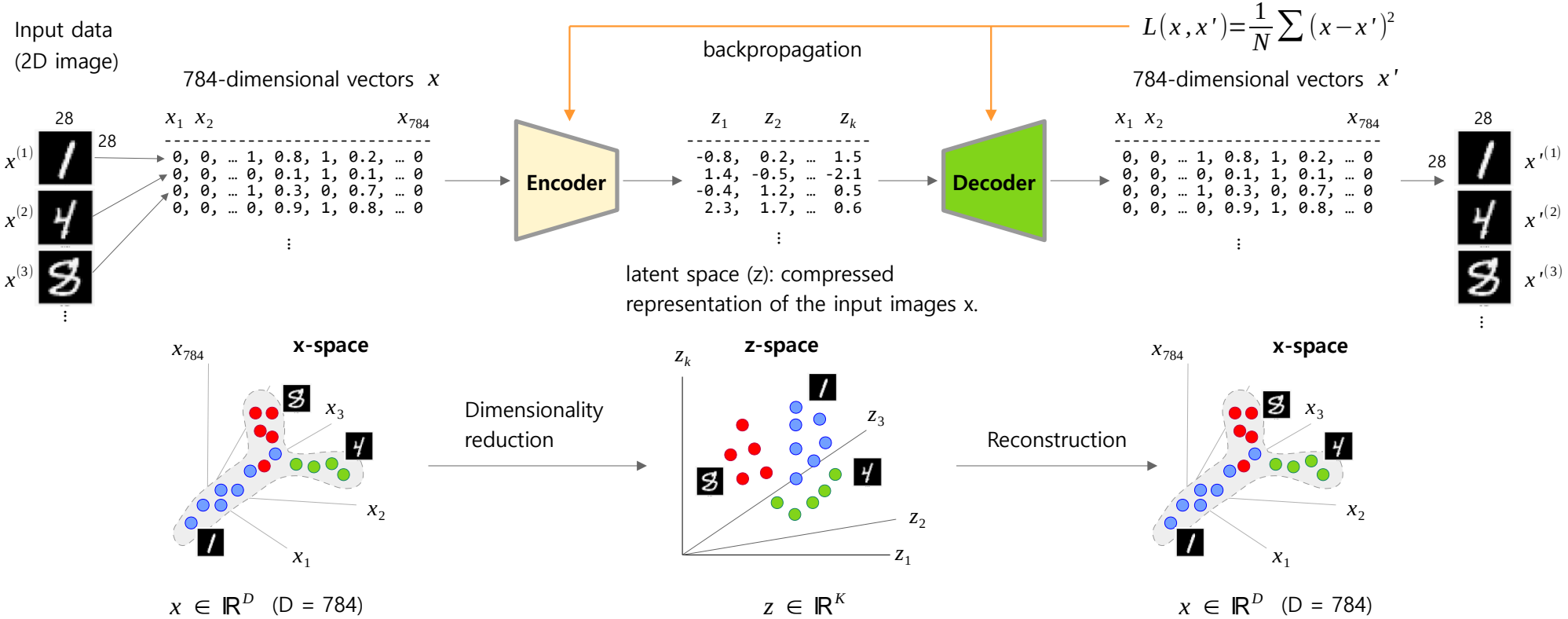


Autoencoder. All rights reserved by www.youtube.com/@meanxai

MX-AI

Dimensionality reduction

- Dimensionality reduction is a method to transform a high-dimensional feature space into a low-dimensional feature space while capturing as many important features of the original data as possible.
- The example below shows the process of encoding MNIST images into k-dimensional vectors and then decoding them into the original images.



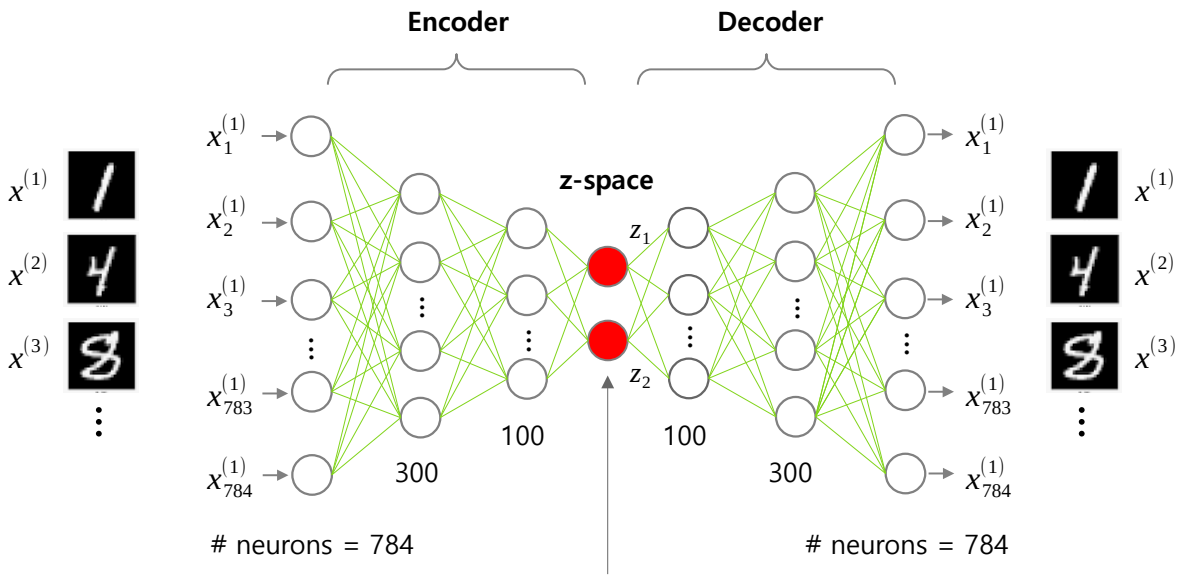
Autoencoder. All rights reserved by www.youtube.com/@meanxai

MX-AI

An example of an autoencoder model for dimensionality reduction

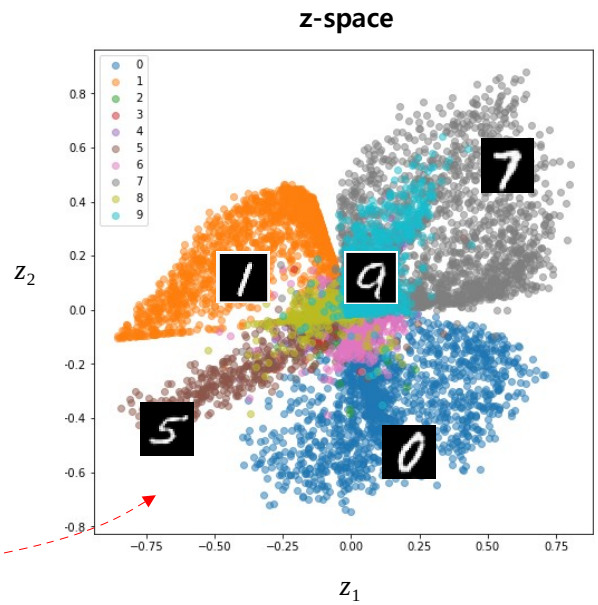
- We are going to build an autoencoder model for dimensionality reduction, consisting of multi-layered fully-connected networks , as shown in the example below. The figure on the right shows the distribution of the latent vectors of x in two-dimensional space.

In z-space, similar images are clustered together.



Latent vectors of x

Compressed representations of the input images x.



2-dimensional space

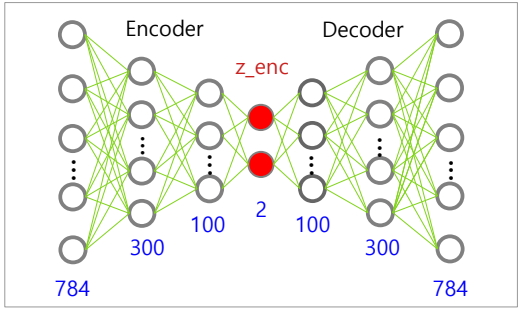
Autoencoder. All rights reserved by www.youtube.com/@meanxai

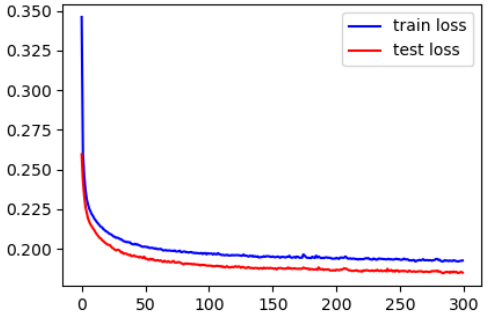
MX-AI

- Implementation of an autoencoder model for dimensionality reduction.

```python
# [MXDL-13-01] 1.dim_reduction(1).py
# Implement an autoencoder model for dimensionality reduction,
# and compare this with the results by principal component
# analysis (PCA).
import numpy as np
from tensorflow.keras.layers import Input, Dense, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.regularizers import L2
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import pickle

# Read an MNIST dataset
with open('data/mnist.pkl', 'rb') as f:
      x, y = pickle.load(f)
x_train, x_test, y_train, y_test = train_test_split(x, y)

# Build an autoencoder model for dimensionality reduction
# Encoder
x_input = Input(batch_shape = (None, x_train.shape[1]))
h_enc = Dense(300, activation = 'relu')(x_input)
h_enc = Dropout(0.5)(h_enc)
h_enc = Dense(100, activation = 'relu')(h_enc)
h_enc = Dropout(0.5)(h_enc)
z_enc = Dense(2, activation = 'linear',
              activity_regularizer = L2(0.01))(h_enc)

# Decoder
h_dec = Dense(100, activation = 'relu')(z_enc)
h_dec = Dropout(0.5)(h_dec)
```

```python
h_dec = Dense(300, activation = 'relu')(h_dec)
h_dec = Dropout(0.5)(h_dec)
x_dec = Dense(x_train.shape[1], activation = 'sigmoid')(h_dec)
model = Model(x_input, x_dec)
model.compile(loss = 'binary_crossentropy',
              optimizer = Adam(learning_rate=0.001))
encoder = Model(x_input, z_enc)

# training
hist = model.fit(x_train, x_train, epochs = 300, batch_size = 500,
                 validation_data = [x_test, x_test])

# Loss history
plt.plot(hist.history['loss'], c='blue', label='train loss')
plt.plot(hist.history['val_loss'], c='red', label='test loss')
plt.legend()
plt.show()
```



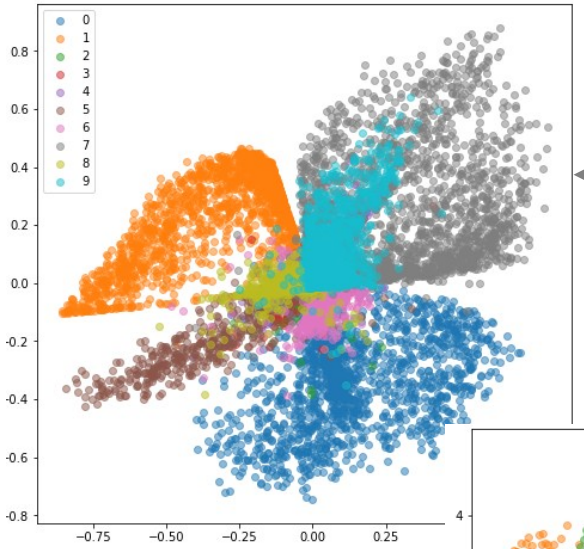


Autoencoder. All rights reserved by www.youtube.com/@meanxai

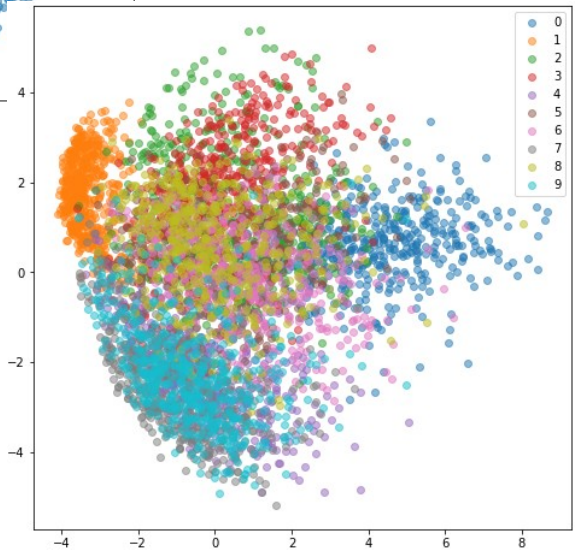
MX-AI

- Implementation of an autoencoder model for dimensionality reduction.

```python
# Reduce the dimensionality of the test data.
x_pred = encoder.predict(x_test)

# z-space
def show_image(x, y):
    plt.figure(figsize=(8,8))
    for i in np.unique(y):
        idx = np.where(y == i)[0]
        z = x[idx]
        plt.scatter(z[:, 0], z[:, 1], alpha=0.5, label=str(i))
    plt.legend()
    plt.show()

print('\nDimensionality reduction by Autoencoder', end='')
show_image(x_pred, y_test)

# Dimensionality reduction by PCA
from sklearn.decomposition import KernelPCA

x_train1 = x_train[:10000]
x_test1 = x_test[:5000]
y_test1 = y_test[:5000]

pca = KernelPCA(n_components=2)
pca.fit(x_train1)
x_pred_pca = pca.transform(x_test1)

print('\nDimensionality reduction by PCA', end='')
show_image(x_pred_pca, y_test1)
```



Dimensionality reduction by Autoencoder



Dimensionality reduction by PCA

Autoencoder. All rights reserved by www.youtube.com/@meanxai

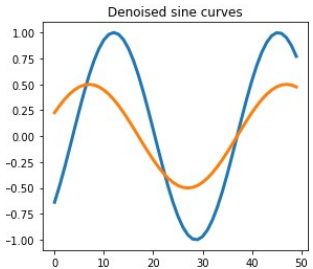
MX-AI

Denoising time series data using LSTM-autoencoder

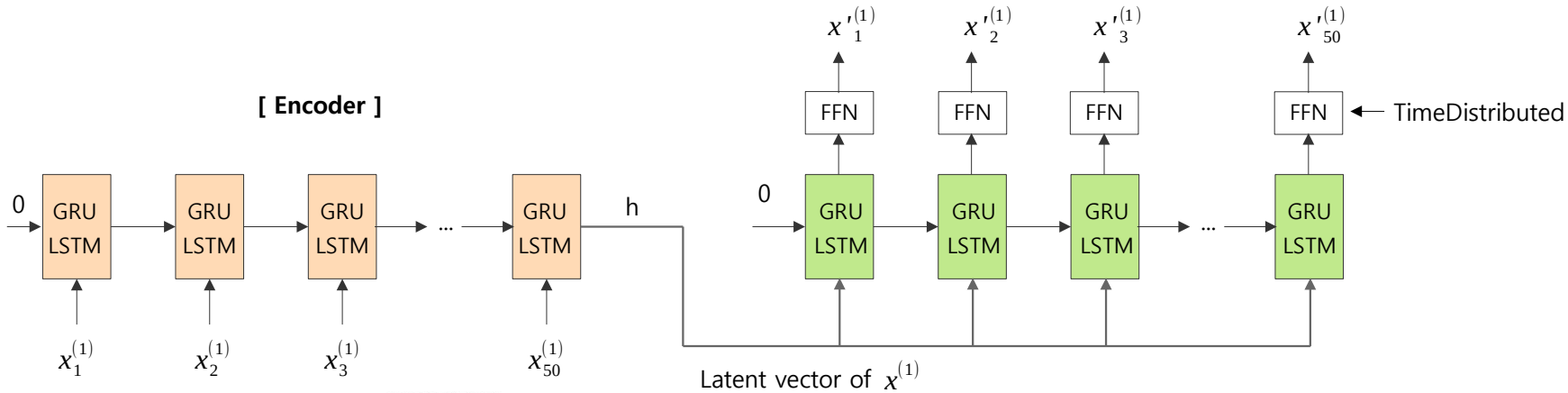
- LSTM autoencoders can be used to remove noise from time series data.
- The structure of a LSTM-autoencoder model is as follows. Both the encoder and decoder consist of LSTMs.

$x'^{(1)}$

t	x_1	x_2
1	0.61,	0.82
2	0.71,	0.88
3	1.23,	0.59
...		
50	1.22,	0.67



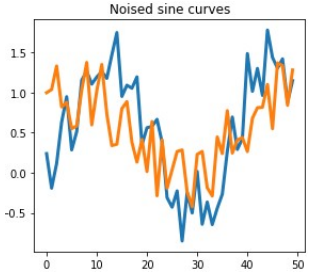
De-noised time series



Noisy time series

$x^{(1)}$

t	x_1	x_2
1	0.63,	0.73
2	0.78,	0.98
3	1.33,	0.73
...		
50	1.12,	0.74



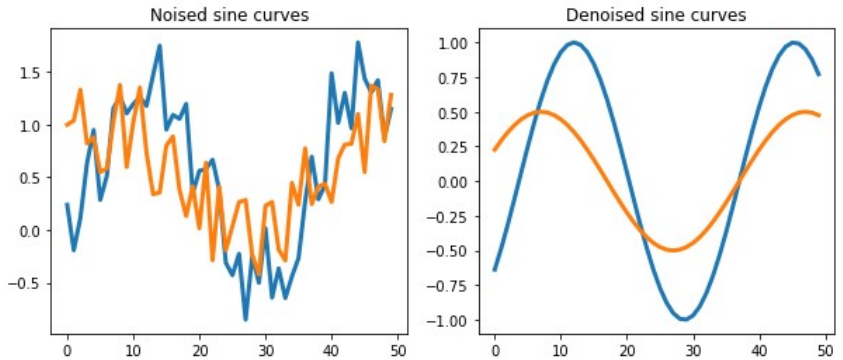
Autoencoder. All rights reserved by www.youtube.com/@meanxai

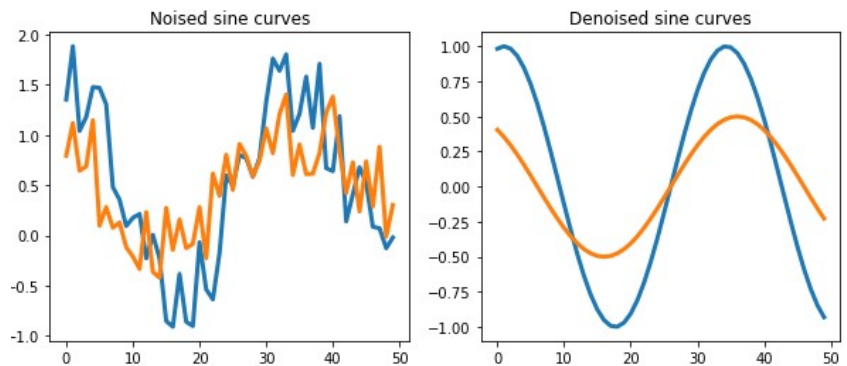
MX-AI

■ Generating noisy time series data

```python
# [MXDL-13-02] 5.noisy_timeseries.py
import numpy as np
import matplotlib.pyplot as plt
import pickle

# Generate training data: 2 noisy sine curves
n = 10000       # the number of data points
n_step = 50     # the number of time steps

# Generate two sine curves
s1 = np.sin(np.pi * 0.06 * np.arange(n))
s2 = 0.5*np.sin(np.pi * 0.05 * np.arange(n))
s = np.vstack([s1, s2]).T  # shape = (1000, 2)

# Add noise to two sine curves
x1 = s1 + np.random.random(n)
x2 = s2 + np.random.random(n)
x = np.vstack([x1, x2]).T  # shape = (1000, 2)

# Generate training data for the LSTM denoising autoencoder
# x_data: noised sine curves
# y_data: noise-free sine curves
# x_data, y_data.shape = (970, 30, 2)
m = np.arange(0, n - n_step)
x_data = np.array([x[i:(i+n_step), :] for i in m])
y_data = np.array([s[i:(i+n_step), :] for i in m])

# Visualize the data with and without noise
i = np.random.choice(x_data.shape[0], 1)[0]
fig, ax = plt.subplots(1, 2, figsize=(10,4))
ax[0].plot(x_data[i], linewidth=3)
ax[0].set_title("Noised sine curves")
ax[1].plot(y_data[i], linewidth=3)
ax[1].set_title("Denoised sine curves")
plt.show()
```

```python
# Save the data with and without noise
with open('data/noisy_sine.pkl', 'wb') as f:
      pickle.dump([x_data, y_data], f)
```





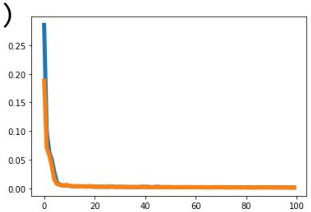
Autoencoder. All rights reserved by www.youtube.com/@meanxai

MX-AI

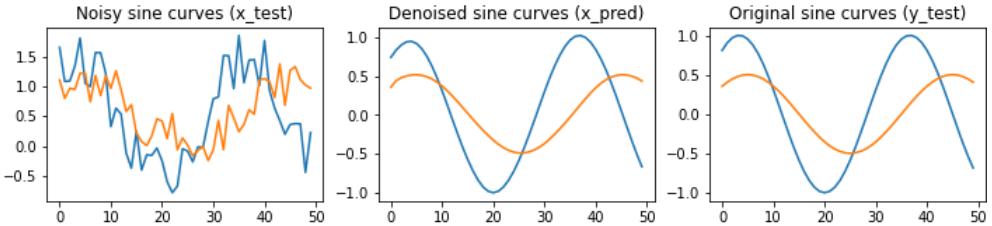
■ Denoising time series data using LSTM-autoencoder

```python
# [MXDL-13-02] 6.denoise_timeseries.py
import numpy as np
from tensorflow.keras.layers import Input, LSTM, RepeatVector
from tensorflow.keras.layers import TimeDistributed, Dense
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
import pickle

# Read noisy time series and split them into training and test data.
# The shape of x, y: (9950, 50, 2)
with open('data/noisy_sine.pkl', 'rb') as f:
        x, y = pickle.load(f)

x_train, x_test, y_train, y_test = train_test_split(x, y)
n_timesteps = x.shape[1]
n_features = x.shape[2]

# Build a denoising LSTM autoencoder model
x_input = Input(batch_shape=(None, *x_train.shape[1:]))
h_enc = LSTM(100)(x_input)                          # Encoder
h_enc = RepeatVector(n_timesteps)(h_enc)
h_dec = LSTM(100, return_sequences=True)(h_enc)  # Decoder
y_output = TimeDistributed(Dense(n_features))(h_dec)

model = Model(x_input, y_output)
model.compile(loss='mse', optimizer=Adam(lr=0.005))
h = model.fit(x_train, y_train, epochs = 100, batch_size=300,
              validation_data=[x_test, y_test])

plt.plot(h.history['loss'])
plt.plot(h.history['val_loss'])
plt.show()
```

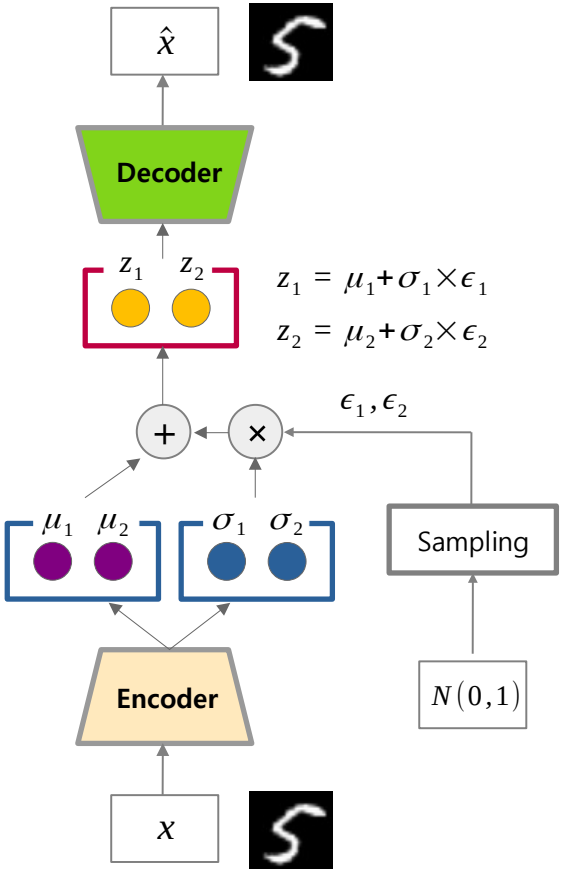
```python
# Denoising x_test and visualizing the results
n = 5
x_pred = model.predict(x_test, verbose=0)
idx = np.random.choice(x_test.shape[0], n)[0]
fig, ax = plt.subplots(n, 3, figsize=(12, 10))

for i in range(n):
    ax[i, 0].plot(x_test[i])
    ax[i, 1].plot(y_test[i])
    ax[i, 2].plot(y_test[i])

    if i == 0:
        ax[i, 0].set_title("Noisy sine curves (x_test)")
        ax[i, 1].set_title("Denoised sine curves (x_pred)")
        ax[i, 2].set_title("Clean sine curves (y_test)")
plt.show()
```



```
 Layer (type)                 Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 50, 2)]           0
 lstm (LSTM)                  (None, 100)              41200
 repeat_vector (RepeatVector)  (None, 50, 100)         0
 lstm_1 (LSTM)                (None, 50, 100)          80400
 time_distributed (TimeDistributed)  (None, 50, 2)     202
```



Autoencoder. All rights reserved by www.youtube.com/@meanxai



13. Autoencoder

Part 5: Variational Autoencoder (VAE)

This video was produced in Korean and translated into English, and the audio was generated by AI (TTS).

www.youtube.com/@meanxai

Autoencoder. All rights reserved by www.youtube.com/@meanxai

MX-AI

Auto-Encoding Variational Bayes

Diederik P. Kingma
Machine Learning Group
Universiteit van Amsterdam
dpkingma@gmail.com

Max Welling
Machine Learning Group
Universiteit van Amsterdam
welling.max@gmail.com

Abstract

How can we perform efficient inference and learning in directed probabilistic models, in the presence of continuous latent variables with intractable posterior distributions, and large datasets? We introduce a stochastic variational inference and learning algorithm that scales to large datasets and, under some mild differentiability conditions, even works in the intractable case. Our contributions are two-fold. First, we show that a reparameterization of the variational lower bound yields a lower bound estimator that can be straightforwardly optimized using standard stochastic gradient methods. Second, we show that for i.i.d. datasets with continuous latent variables per data point, posterior inference can be made especially efficient by fitting an approximate inference model (also called a recognition model) to the intractable posterior using the proposed lower bound estimator. Theoretical advantages are reflected in experimental results.

3. Example: Variational Auto-Encoder

In this section we'll give an example where we use a neural network for the probabilistic encoder $q_\Phi(z|x)$ the approximation to the posterior of the generative model $p_\theta(x,z)$) and where the parameters and are optimized jointly with the AEVB algorithm.
Let the prior over the latent variables be the centered isotropic multivariate Gaussian $p_\theta(z)=N(z; 0, I)$. Note that in this case, the prior lacks parameters. We let $p_\theta(x|z)$ be a multivariate Gaussian (in case of real-valued data) or Bernoulli (in case of binary data) whose distribution parameters are computed from z with a MLP (a fully-connected neural network with a single hidden layer, see appendix C). Note the true posterior $p_\theta(z|x)$ is in this case intractable. While there is much freedom in the form $q_\Phi(z|x)$, we'll assume the true (but intractable) posterior takes on an approximate Gaussian form with an approximately diagonal covariance. In this case, we can let the variational approximate posterior be a multivariate Gaussian with a diagonal covariance structure[2]:

$$\log q_\phi(z|x^{(i)}) = \log N(z; \mu^{(i)}, \sigma^{2(i)} I)$$

where the mean and s.d. of the approximate posterior, $\mu^{(i)}$ and $\sigma^{(i)}$, are outputs of the encoding MLP ,i.e. nonlinear functions of data point $x^{(i)}$ and the variational parameters (see appendix C). As explained in section 2.4, we sample from the posterior $z^{(i,l)} \sim q_\Phi(z|x^{(i)})$ using $z^{(i,l)}=g(x(i), \varepsilon(l))=\mu^{(i)}+\sigma^{(i)}\odot\varepsilon^{(l)}$ where $\varepsilon^{(l)} \sim N(0, I)$. With $\odot$ we signify an element-wise product. In this model both $p_\theta(z)$ (the prior) and $q_\Phi(z|x)$ are Gaussian; in this case, we can use the estimator of eq.(7) where the KL divergence can be computed and differentiated without estimation (see appendix B). The resulting estimator for this model and data point $x^{(i)}$ is:

$$L(\theta, \phi; x^{(i)}) \approx \frac{1}{2}\sum_{j=1}^{J}\left(1+\log((\sigma_j^{(i)})^2)-(\mu_j^{(i)})^2-(\sigma_j^{(i)})^2\right)+\frac{1}{L}\sum_{l=1}^{L}\log p_\theta(x^{(i)}|z^{(i,l)})$$

where $z^{(i,l)}=\mu^{(i)}+\sigma^{(i)}\odot\epsilon^{(l)}$ and $\epsilon^{(l)}\sim N(0,I)$

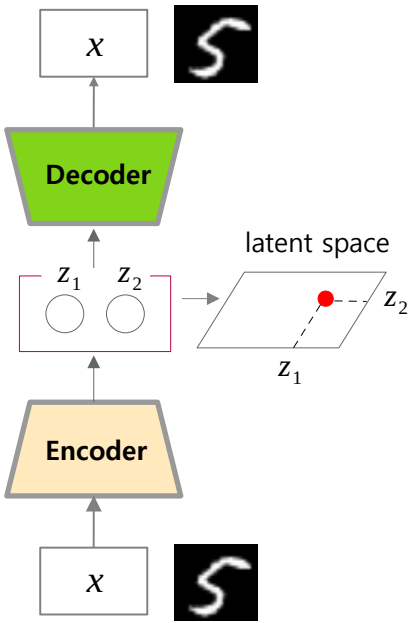
Autoencoder. All rights reserved by www.youtube.com/@meanxai

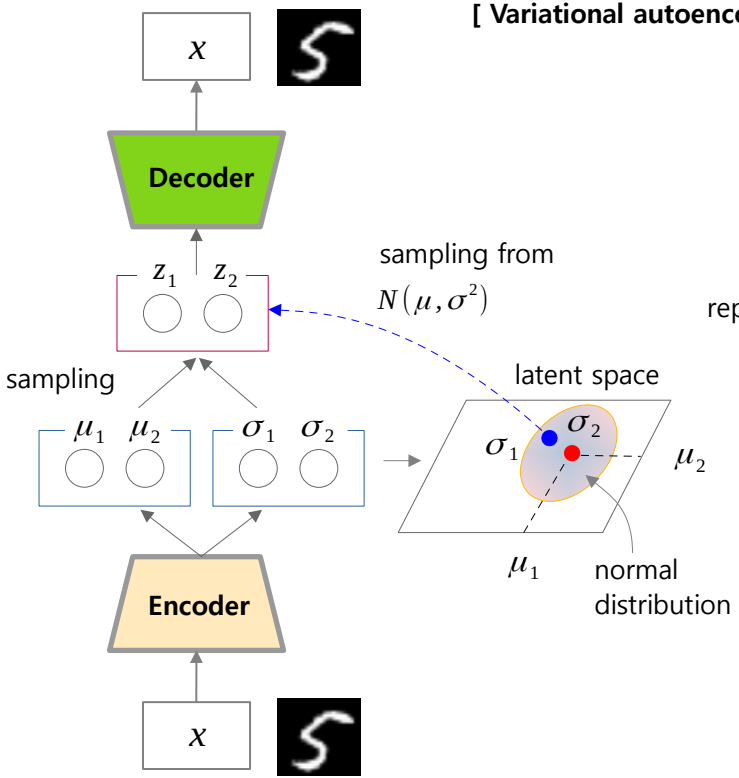
MX-AI

Overview of Variational Autoencoder (VAE)

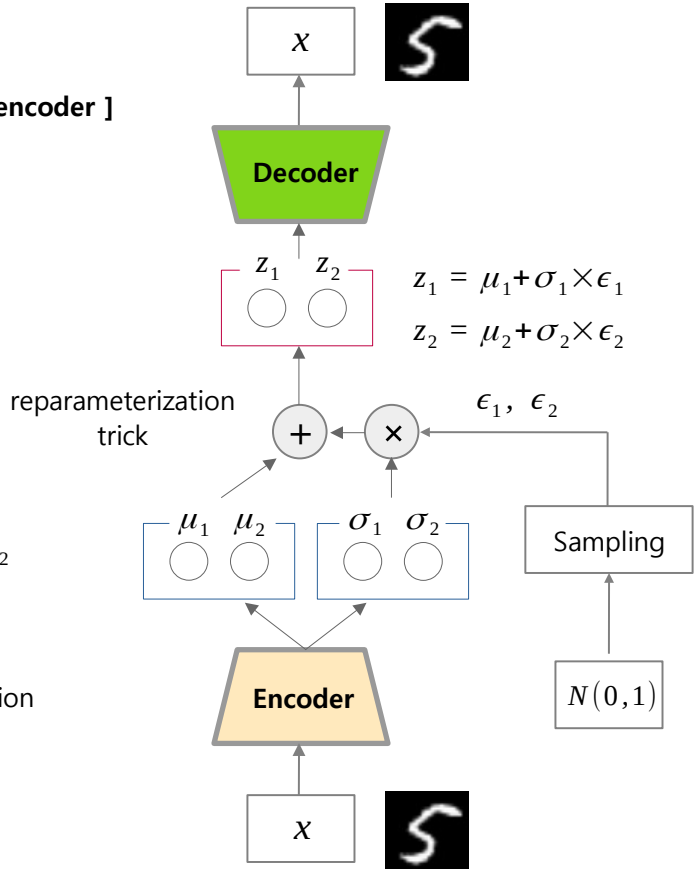
- For a single input, the encoder of an autoencoder outputs a latent vector in the latent space, whereas the encoder of a VAE outputs a distribution (usually Gaussian) in the latent space. The decoder samples a point from this distribution in latent space and reconstructs the original input.

[Standard autoencoder]





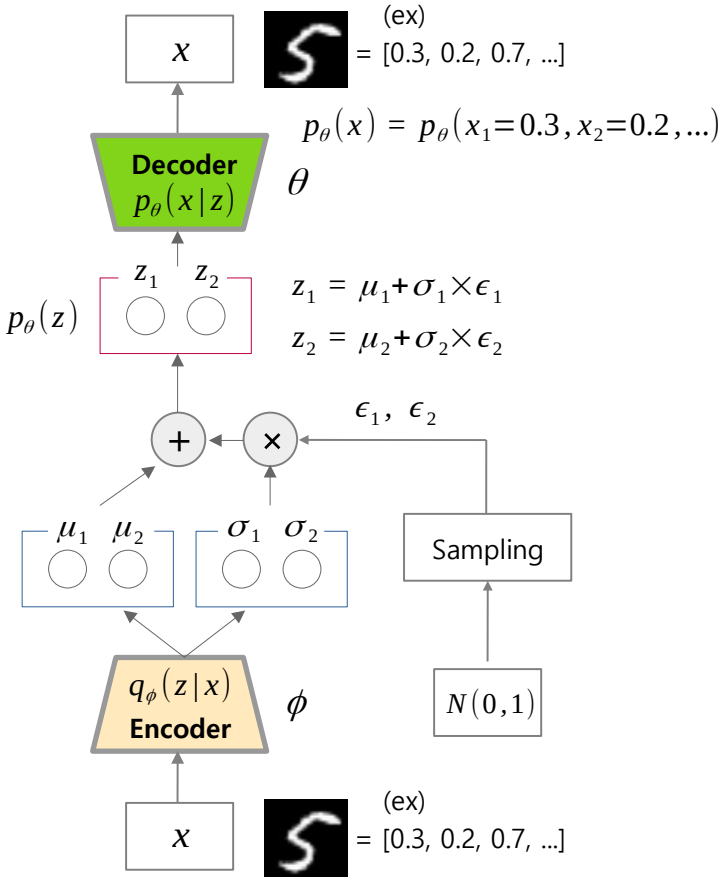
[Variational autoencoder]



Autoencoder. All rights reserved by www.youtube.com/@meanxai

MX-AI

Loss function for VAE



Goal: We want to estimate the true parameters θ*, Φ* such that $p_\theta(x)$ is maximized.

$$\max_{\theta,\phi} p_\theta(x)$$

$$p_\theta(x) = \int_z p_\theta(x,z)\,dz$$ ← marginal distribution of x

$$p_\theta(x,z) = p_\theta(z)\,p_\theta(x|z)$$ ← The Bayes' theorem

$$p_\theta(x) = \int_z p_\theta(z)\,p_\theta(x|z)\,dz$$ ← marginal likelihood (intractable)

Gaussian distribution as a prior

Neural network (Decoder)

$$p_\theta(x) = \frac{p_\theta(z)\,p_\theta(x|z)}{p_\theta(z|x)}$$ ← the true posterior density

intractable

How can we express the p theta of x in a tractable way?
To this end, let's introduce a recognition model $q_\Phi(z|x)$ an approximation to the intractable true posterior $p_\theta(z|x)$.

$$q_\phi(z|x) \simeq p_\theta(z|x)$$

tractable (variational approximation)

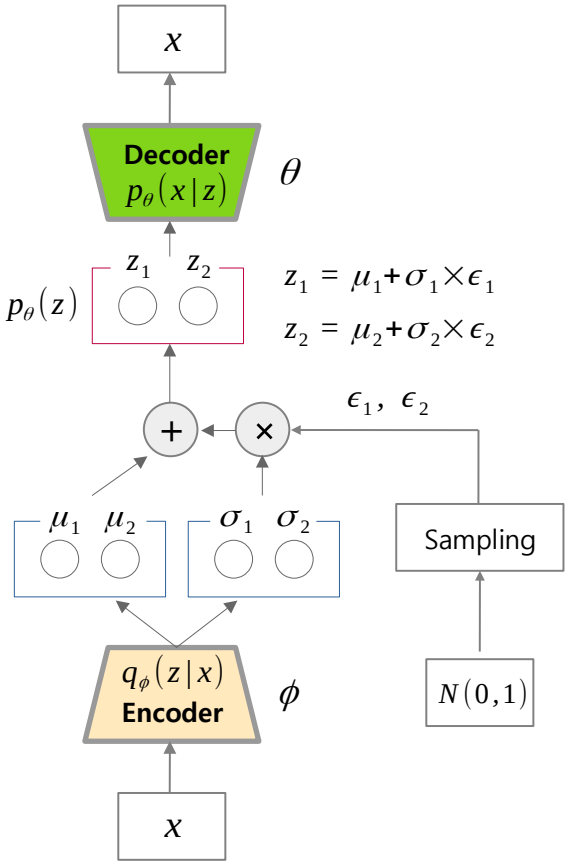
intractable

$$p_\theta(x) \simeq \frac{p_\theta(z)\,p_\theta(x|z)}{q_\phi(z|x)}$$ ← tractable

Autoencoder. All rights reserved by www.youtube.com/@meanxai

MX-AI

Loss function for VAE



Goal: $\max_{\theta,\phi} p_\theta(x)$ or $\max_{\theta,\phi} \log p_\theta(x)$

$$p_\theta(z|x) = \frac{p_\theta(x|z)\,p_\theta(z)}{p_\theta(x)} = \frac{p_\theta(x,z)}{p_\theta(x)}$$

$$D_{KL}(q_\phi(z|x)\|p_\theta(z|x)) = -\sum_z q_\phi(z|x)\log\frac{p_\theta(z|x)}{q_\phi(z|x)}$$

$$= -\sum_z q_\phi(z|x)\left[\log\frac{p_\theta(x,z)}{q_\phi(z|x)} - \log p_\theta(x)\right]$$

$$= -\sum_z q_\phi(z|x)\log\frac{p_\theta(x,z)}{q_\phi(z|x)} + \log p_\theta(x)\sum_z q_\phi(z|x)$$

$$= -\sum_z q_\phi(z|x)\log\frac{p_\theta(x,z)}{q_\phi(z|x)} + \log p_\theta(x)$$

$$\log p_\theta(x) = D_{KL}(q_\phi(z|x)\|p_\theta(z|x)) + \boxed{\sum_z q_\phi(z|x)\log\frac{p_\theta(x,z)}{q_\phi(z|x)}} \quad \text{---- (1)}$$

variational lower bound

$$\log p_\theta(x) \geq \sum_z q_\phi(z|x)\log\frac{p_\theta(x,z)}{q_\phi(z|x)}$$

$$L = \sum_z q_\phi(z|x)\log\frac{p_\theta(x,z)}{q_\phi(z|x)}$$ ← We can maximize this to achieve our goal.

$$= \sum_z q_\phi(z|x)\log\frac{p_\theta(x|z)\,p_\theta(z)}{q_\phi(z|x)}$$

$$= \sum_z q_\phi(z|x)\log p_\theta(x|z) + \sum_z q_\phi(z|x)\log\frac{p_\theta(z)}{q_\phi(z|x)}$$

$$= E_{q_\phi(z|x)}\log p_\theta(x|z) - D_{KL}(q_\phi(z|x)\|p_\theta(z)) \quad \text{---- (3)}$$

$\max(L)$ or $\min(-L)$

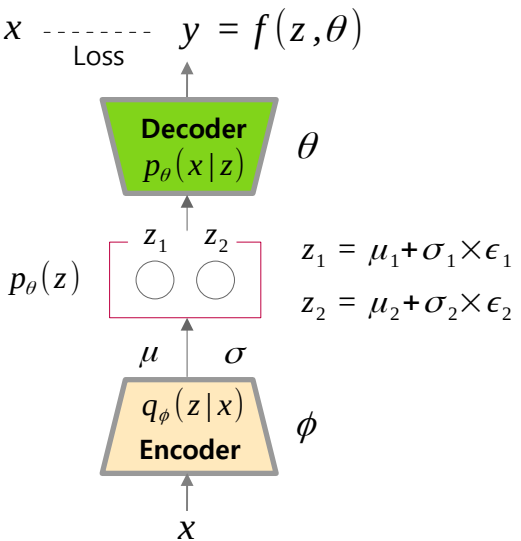
$$\min[-\underbrace{E_{q_\phi(z|x)}\log p_\theta(x|z)}_{\text{Expected reconstruction error}} + \underbrace{D_{KL}(q_\phi(z|x)\|p_\theta(z))}_{\text{regularization term}}]$$

Autoencoder. All rights reserved by www.youtube.com/@meanxai

MX-AI

Loss function for VAE

$$min[-\underbrace{E_{q_\phi(z|x)}\log p_\theta(x|z)}_{\text{Expected reconstruction error}} + \underbrace{D_{KL}(q_\phi(z|x)\|p_\theta(z))}_{\text{regularization}}]$$



$x \in \mathbb{R}^D, \quad z \in \mathbb{R}^K$

Assumptions
$$\begin{cases} p_\theta(z) \sim N(z|0,1) & \leftarrow \text{Standard normal distribution} \\ q_\phi(z|x) \sim N(z|\mu,\sigma^2) & \leftarrow \text{Normal distribution} \end{cases}$$

$$E_{q_\phi(z|x)}\log p_\theta(x|z) \simeq \frac{1}{L}\sum_{l=1}^{L}\log p_\theta(x|z^{(l)}) \simeq \log p_\theta(x|z)$$

Estimation by sampling L times per data point. L per data point can be set to 1 as long as the mini-batch size M is large enough, e.g. M = 100.

$$D_{KL}(P\|Q) = \frac{1}{2}\left[\frac{(\mu_Q-\mu_P)^2}{\sigma_Q^2} + \frac{\sigma_P^2}{\sigma_Q^2} - \log\frac{\sigma_P^2}{\sigma_Q^2} - 1\right]$$: KLD for the normal distribution

$$D_{KL}(q_\phi(z|x)\|p_\theta(z)) = \frac{1}{2}\sum_{i=1}^{K}(\sigma_i^2 + \mu_i^2 - \log(\sigma_i^2) - 1)$$

- MLE for multivariate Gaussian (in case of real-valued data)

$$p_\theta(x|z) \simeq \prod_{i=1}^{D}\exp(-|x_i-y_i|^2)$$

$$\log p_\theta(x|z) \simeq -\sum_{i=1}^{D}|x_i-y_i|^2$$

- MLE for Bernoulli distribution (in case of binary data)

$$p_\theta(x|z) \simeq \prod_{i=1}^{D} y_i^{x_i}(1-y_i)^{1-x_i}$$

$$\log p_\theta(x|z) \simeq \sum_{i=1}^{D} x_i\log y_i + (1-x_i)\log(1-y_i)$$

$$min[\sum_{i=1}^{D}|x_i-y_i|^2 + \frac{1}{2}\sum_{i=1}^{K}(\sigma_i^2+\mu_i^2-\log(\sigma_i^2)-1)]$$

reconstruction loss (MSE) **regularization loss** (KL)

or

$$min[-\sum_{i=1}^{D} x_i\log y_i + (1-x_i)\log(1-y_i) + \frac{1}{2}\sum_{i=1}^{K}(\sigma_i^2+\mu_i^2-\log(\sigma_i^2)-1)]$$

reconstruction loss (binary cross entropy) **regularization loss** (KL)

Autoencoder. All rights reserved by www.youtube.com/@meanxai

MX-AI

Loss function for VAE

- **Stochastic gradient descent:**

Gaussian: $min[\ \sum_{i=1}^{D} |x_i - y_i|^2 + \frac{1}{2}\sum_{i=1}^{K}(\sigma_i^2 + \mu_i^2 - \log(\sigma_i^2) - 1)\ ]$

Bernoulli: $min[\ -\sum_{i=1}^{D} x_i \log y_i + (1-x_i)\log(1-y_i) + \frac{1}{2}\sum_{i=1}^{K}(\sigma_i^2 + \mu_i^2 - \log(\sigma_i^2) - 1)\ ]$

- **Mini-batch gradient descent: (M is mini-batch size)**

Gaussian: $min[\frac{1}{M}\sum_{m=1}^{M}(\sum_{i=1}^{D}|x_i^{(m)} - y_i^{(m)}|^2 + \frac{1}{2}\sum_{i=1}^{K}(\sigma^{(m)2}_i + \mu^{(m)2}_i - \log(\sigma^{(m)2}_i) - 1))]$

Bernoulli: $min[\frac{1}{M}\sum_{m=1}^{M}(-\sum_{i=1}^{D} x_i^{(m)} \log y_i^{(m)} + (1-x_i^{(m)})\log(1-y_i^{(m)}) + \frac{1}{2}\sum_{i=1}^{K}(\sigma^{(m)2}_i + \mu^{(m)2}_i - \log(\sigma^{(m)2}_i) - 1))]$

Autoencoder. All rights reserved by www.youtube.com/@meanxai